GSP_API: A Backend-Agnostic Scene-Description API for Scientific Visualization in Python

Abstract

GSP_API is a Python implementation of a Graphic Server Protocol-style scene-description API for 2D and 3D scientific visualization. It separates the visualization API from the rendering implementation, exposing a single declarative scene model — canvas, viewports, visuals, transforms, materials, geometry, cameras — that can be dispatched to one of three interchangeable renderers: a CPU-bound Matplotlib backend, a GPU-bound Datoviz/Vulkan backend, and a Flask-based network backend that serializes scenes to a remote rendering server. Backend selection is performed at runtime through a static registry rather than through code changes, so the same example script runs against every backend. Scene objects are mirrored in a Pydantic v2 layer that supports JSON serialization and base64-encoded buffers for transport. The system targets two distinct audiences: engineers building plotting tools who need a portability layer between CPU and GPU stacks, and researchers studying rendering-library API design across heterogeneous backends. GSP_API is currently a research prototype at version 0.1.0; it has known bugs, an active TODO list, and no published release. This white paper describes its architecture, its trade-offs, its security posture, and its position relative to existing visualization libraries.

1. Introduction

Scientific visualization in Python has been dominated for over a decade by Matplotlib, whose ubiquity, maturity, and publication-quality output have made it the reference toolkit for static plotting. As datasets have grown — millions of points in single-cell genomics, large simulation grids in climate and physics, multi-resolution image pyramids in microscopy — Matplotlib's CPU-bound rasterization pipeline has become a routine bottleneck. GPU-accelerated alternatives have emerged in response, including Datoviz, VisPy, and various WebGL-based libraries. Each of these resolves the performance problem inside its own walled garden: it ships an API and a renderer as a single, indivisible product. Switching from one library to another typically means rewriting visualization code from the ground up.

A second axis of fragmentation has appeared with the growth of remote and headless workloads. HPC clusters, cloud notebooks, and CI pipelines need rendering that does not depend on a local display server. Browser-targeted libraries (Plotly, Bokeh, deck.gl) address part of this need but at the cost of an entirely different scene model and runtime. The result is a landscape in which the same scientific figure may need to be expressed three times — once in Matplotlib for publication, once in a GPU library for interactive exploration, once in a web library for sharing — with no shared substrate underneath.

GSP_API is a response to that fragmentation. It applies a long-standing idea from graphics systems research, the separation of scene description from rendering, to the specific shape of the scientific Python ecosystem. The repository under discussion is the API layer plus three reference renderers; it is not a competitor to Matplotlib or Datoviz, but a coordination layer above them.

2. Problem Statement

The problem GSP_API addresses can be stated precisely:

> *A Python visualization program written today is structurally bound to a single rendering implementation. To change rendering backends — for performance, deployment, or comparative research — the program must be rewritten.*

This binding has three concrete sources.

API/renderer coupling. Matplotlib, Datoviz, VisPy, Plotly, and Bokeh each define their own scene model: Matplotlib's `Figure` / `Axes` / `Artist` hierarchy, Datoviz's app/figure/panel/visual model, Plotly's JSON figure schema. These are not interchangeable, and no library publishes a separable scene-description layer.

Performance/portability trade-off. Matplotlib runs everywhere but its Agg rasterizer scales poorly past roughly 100k primitives. Datoviz delivers GPU performance through Vulkan but requires a working Vulkan driver and is not currently a portable abstraction across operating systems and headless environments. VisPy sits between the two but couples its scene graph to its OpenGL canvas.

Local/remote split. Remote rendering — sending a scene to a server and receiving back an image or stream — is generally implemented as a separate stack with its own API surface (notebook tunnels, web frameworks, custom protocols). It is rarely treated as one renderer choice among others.

Existing solutions partially address these axes but do not span all three:

- Matplotlib provides reach but no GPU path.
- Datoviz provides GPU performance but no remote path and limited OS portability.
- VisPy provides multi-canvas rendering but inside a single GL-coupled API.
- Plotly and Bokeh provide remote/web rendering but only for their own scene models.
- Generic IPC tools (Jupyter widgets, RPC servers) move bytes but do not standardize the scene.

GSP_API treats the scene as the invariant and the renderer as the variable.

3. Proposed Solution

GSP_API contributes one thing: a backend-agnostic Python API for describing 2D and 3D scientific scenes, paired with a registry-based dispatch mechanism that lets any number of renderers implement that API.

The core scene model has six concept families:

- **Core objects.** `Canvas` (dimensions, DPI, background color), `Viewport` (rectangular region inside a canvas with its own background), `Camera` (view and projection matrices), `Texture`, `Event`.
- **Visual primitives.** Eight subclasses of `VisualBase`: `Points`, `Markers`, `Segments`, `Paths`, `Texts`, `Pixels`, `Image`, `Mesh`.
- **Geometry.** `Geometry` and `MeshGeometry` carrying typed vertex and index buffers.
- **Transforms.** Composable `TransformChain` pipelines with serializable links.
- **Materials.** Extensible material descriptors attached to visuals.
- **Types.** Typed buffers (`Buffer`, `BufferType`), the `TransBuf` union (`Buffer | TransformChain`), and abstract bases (`VisualBase`, `RendererBase`, `AnimatorBase`, `ViewportEventsBase`).

Three renderer packages implement the same renderer contract:

- `gsp_matplotlib` — CPU rendering via Matplotlib.
- `gsp_datoviz` — GPU rendering via Datoviz on Vulkan.
- `gsp_network` — Flask-based remote rendering (the client serializes the scene; the server runs `gsp_matplotlib` or `gsp_datoviz` and returns a PNG).

Selection is performed at runtime by a string name: `RendererRegistry.create_renderer("matplotlib"|"datoviz"|"network", canvas)`. Switching renderers is a one-line change.

4. System Architecture

4.1 Components

The repository is organized as a multi-package layout under `src/` :

```
src/
├── gsp/                  Core scene API (no renderer dependencies)
│   ├── core/             Canvas, Viewport, Camera, Texture, Event
│   ├── types/            VisualBase, RendererBase, AnimatorBase,
│   │                     ViewportEventsBase, Buffer, TransBuf
│   ├── visuals/          Eight visual primitive classes
│   ├── geometry/         Geometry, MeshGeometry
│   ├── transforms/       TransformChain
│   ├── materials/        Material base classes
│   └── utils/            renderer_registery.py, UUID utilities
├── gsp_matplotlib/       Matplotlib backend
├── gsp_datoviz/          Datoviz backend
├── gsp_network/          Flask client/server renderer
├── gsp_pydantic/         Pydantic mirror types and serializer
├── gsp_extra/            Auxiliary helpers
├── gsp_nico/             Collaborator sandbox
└── vispy2/               Higher-level VisPy-style API on top of gsp
```

The core package `gsp` has no dependency on any renderer; renderers depend on `gsp` . The Pydantic layer is segregated into `gsp_pydantic` so that `gsp` itself does not require Pydantic at import time.

4.2 Data flow

For a local renderer (Matplotlib or Datoviz):

```
User code
  │
  ├── builds Canvas, Viewports, Visuals, Cameras, model matrices
  │
  ▼
RendererRegistry.create_renderer(name, canvas)
  │
  ▼
RendererBase.render(viewports, visuals, model_matrices, cameras) → bytes
  │
  ├── Matplotlib: draws into per-viewport Axes and Artists
  └── Datoviz:    submits to dvz.App / dvz.Figure / dvz.Panel / dvz.Visual
```

For the network renderer:

```
Client (NetworkRenderer)
  │
  ├── PydanticSerializer.serialize(scene) → PydanticDict
  │       (buffers base64-encoded, visuals as discriminated union)
  │
  ├── HTTP POST /render
```

```
  │       body = NetworkPayload {
  │           renderer_name: "matplotlib" | "datoviz",
  │           data:          PydanticDict
  │       }
  │
  ▼
Server (tools/network_server.py, Flask)
  │
  ├── PydanticParser.parse(payload.data) → live GSP objects
  ├── chosen renderer.render(...) → PNG bytes
  │       (Datoviz used in offscreen=True mode)
  │
  ▼
Client receives PNG, blits into local Matplotlib figure
   with source-over alpha compositing
```

4.3 Common contract

All renderers implement `RendererBase` (`src/gsp/types/renderer_base.py`):

```python
class RendererBase(ABC):
    @abstractmethod
    def __init__(self, canvas: Canvas): ...
    @abstractmethod
    def render(
        self,
        viewports: Sequence[Viewport],
        visuals: Sequence[VisualBase],
        model_matrices: Sequence[TransBuf],
        cameras: Sequence[Camera],
    ) -> bytes: ...
    @abstractmethod
    def show(self) -> None: ...
    @abstractmethod
    def close(self) -> None: ...
    @abstractmethod
    def get_canvas(self) -> Canvas: ...
    @abstractmethod
    def clear(self) -> None: ...
```

Two parallel abstract bases govern interactivity and time:

- `AnimatorBase` — frame-callback registration and a `start()` method that sweeps over viewports, visuals, model matrices, and cameras.
- `ViewportEventsBase` — keyboard, mouse, and resize events as typed `Event[Callback]` channels.

Each backend registers all three component types together:

```python
RendererRegistry.register_renderer(
    renderer_name="matplotlib",
    renderer_base_type=MatplotlibRenderer,
```

```
    viewport_event_base_type=ViewportEventsMatplotlib,
    animator_base_type=AnimatorMatplotlib,
)
```

4.4 Key design decisions

Registry over environment variable. Backend selection is a string passed to `RendererRegistry.create_renderer`. There is no `GSP_RENDERER` import-time switch; the choice is explicit at the call site, which makes mixed-backend programs (e.g., one figure on Matplotlib, another on Datoviz) trivial.

Dual core/Pydantic layer. Core scene objects are plain Python classes; Pydantic v2 mirror types live in `gsp_pydantic`. This keeps the core dependency footprint small and lets the network path serialize without the core having to know about it.

UUID-based identity. Every `VisualBase`, `Canvas`, and `Viewport` carries a UUID. Renderers key their internal state on these UUIDs (e.g., `_axes: dict[str, matplotlib.axes.Axes]` in `gsp_matplotlib`, `_dvz_panels: dict[str, _DvzPanel]` in `gsp_datoviz`). This is the substrate on which incremental updates can later be implemented.

`TransBuf` union. Visuals accept either a raw `Buffer` or a `TransformChain` for any data slot. This allows lazy data pipelines to be expressed declaratively in the scene instead of materialized eagerly in NumPy.

Event pub/sub. Input handling is decoupled from rendering through a generic `Event[Callback]` mechanism with `subscribe`, `unsubscribe`, and `dispatch` operations and an `@event_listener` decorator.

5. Key Features

5.1 Backend-agnostic scene API

A user writes against `Canvas`, `Viewport`, `Camera`, `Visual` subclasses, `TransformChain`, and `Material`. No backend symbol appears in user code. The trade-off is feature ceiling: features that exist only on one backend (e.g., Datoviz GPU specials) require either an escape hatch or a feature-parity workaround. The current API stays close to the lowest common denominator.

5.2 Eight visual primitives

`Points`, `Markers`, `Segments`, `Paths`, `Texts`, `Pixels`, `Image`, `Mesh`. All inherit `VisualBase` and carry `userData: dict[str, Any]` for application-level metadata. Trade-off: backends do not have full feature parity (text anchor and rotation are currently broken in the Datoviz backend; see Section 10).

5.3 Pluggable renderers via static registry

Backends register at import time. Adding a fourth renderer is a matter of subclassing `RendererBase`, `AnimatorBase`, and `ViewportEventsBase` and calling `RendererRegistry.register_renderer`. Trade-off: registration depends on import order; there is no plugin-discovery mechanism (entry points), so users must import the backend package once before constructing.

5.4 Pydantic-typed scene with base64 buffers

`gsp_pydantic` provides `PydanticCanvas`, `PydanticViewport`, `PydanticBuffer` (with base64 binary payload), `PydanticCamera`, and a discriminated `PydanticVisual` union over the eight primitives. `PydanticSerializer.serialize` and `PydanticParser.parse` round-trip scenes between live objects and JSON dicts. Trade-off: base64 inflates payloads by ~33%, and there is no diff protocol; every render call sends the full scene.

5.5 Shared animator and viewport-event subsystems

Both `Animator` and `ViewportEvents` are abstract in core and implemented per backend. The shared abstract surface lets animation and interactivity code remain renderer-agnostic. Trade-off: the shared event model is necessarily a lowest-common-denominator (e.g., touch events, GPU-specific gestures, or framebuffer-level callbacks are not exposed).

5.6 Flask-based network renderer

`gsp_network.NetworkRenderer` (client) and `tools/network_server.py` (Flask server) implement remote rendering with a single `POST /render` endpoint. The client serializes the scene through `PydanticSerializer`, the server reconstructs and renders into PNG, and the client composes the PNG into a local Matplotlib figure. Trade-off: per-render HTTP round-trip latency and full-scene re-serialization on every frame; suitable for offline or low-frame-rate workflows, not interactive 60 Hz rendering.

5.7 Documentation and example gallery

The repository ships a Material-themed mkdocs site (`mkdocs.yml`, `mkdocs_source/`) with seven API reference pages (`gsp`, `gsp_network`, `gsp_pydantic`, `gsp_matplotlib`, `gsp_datoviz`, `gsp_extra`, `vispy_2`) and an `examples/` directory with on the order of 58 production-ready examples plus roughly 17 work-in-progress examples (underscore-prefixed). Trade-off: the docs build is local-only; no public docs URL is configured.

6. Technical Deep Dive

6.1 Renderer registry

```
RendererRegistry
  _registry: dict[str, RendererRegistryItem]
                    ↑
                    └── RendererRegistryItem {
                            renderer_base_type,
                            viewport_event_base_type,
                            animator_base_type
                        }

  register_renderer(name, renderer, viewport_events, animator) → None
  create_renderer(name, canvas)                                 → RendererBase
  create_animator(name, ...)                                    → AnimatorBase
  create_viewport_events(name, ...)                             → ViewportEventsBase
```

Backends populate the registry at import time via their `renderer_registration.py` modules. Resolution is by string name.

6.2 Scene data model

The `TransBuf` union (`Buffer | TransformChain`) is the workhorse of GSP_API's data pipeline:

- `Buffer` is a typed bytearray. The element type is described by a `BufferType` enum that defines item size and dtype; the count is fixed at construction but contents are mutable.
- `TransformChain` is a sequence of `TransformLinkBase` objects. `TransformChain.run()` returns a `Buffer` . Chains are fully serializable (transform composition crosses the network alongside the scene).

A `Camera` holds two `TransBuf` matrices (view and projection); a `Mesh` visual carries vertex and index buffers as `Buffer` ; positions and colors of `Points` and `Markers` are `TransBuf` , allowing computed inputs.

6.3 Datoviz mapping

The Datoviz backend translates GSP concepts into Datoviz objects:

GSP concept	Datoviz concept
`Canvas`	`dvz.App` + `dvz.Figure`
`Viewport`	`dvz.Panel`
`VisualBase`	`dvz.Visual`
Texture	`dvz.Texture`

It maintains three UUID-keyed dictionaries (`_dvz_panels` , `_dvz_visuals` , `_dvz_textures`) to reconcile incoming GSP scenes with existing Datoviz state. The backend supports `offscreen=True` , which is what the network server uses to render Datoviz scenes without a display.

6.4 Network protocol

Endpoint: `POST /render` . Request body:

```
class NetworkPayload(TypedDict):
    renderer_name: Literal["matplotlib", "datoviz"]
    data: PydanticDict
```

The server selects the backend by `renderer_name` , reconstructs the scene with `PydanticParser.parse` , instantiates the chosen `RendererBase` (Datoviz with `offscreen=True`), renders to PNG, and returns the bytes. The client (`NetworkRenderer`) draws the PNG into a local Matplotlib figure with source-over alpha compositing, which allows multiple `render` calls to layer onto a single client canvas — though the current implementation re-renders the whole scene server-side on every call.

6.5 Performance considerations

The three backends define three performance regimes:

- **Matplotlib (CPU).** Bound by the Agg rasterizer. Suitable for tens of thousands of primitives at interactive rates; degrades smoothly past that. Bottleneck: per-primitive Python and rasterization overhead.
- **Datoviz (GPU).** Bound by Vulkan driver and PCIe upload. Suitable for millions of primitives at 60 Hz on commodity GPUs. Bottleneck: buffer upload and shader compilation; cold-start latency is non-

trivial.
- **Network.** Bound by serialization and HTTP RTT. Each `render` call serializes the full scene, transports it, and decodes a PNG; there is no incremental update or streaming protocol. Suitable for offline rendering, snapshot generation, or notebooks talking to a remote GPU.

No benchmarks are published in the repository; the regime characterization above is qualitative.

6.6 Scalability

The architectural ceiling on scalability is the absence of an incremental update protocol. Today, every `render` call is a full re-render. Two facets of the existing design point toward incremental updates:

- UUID-keyed visual identity, already used by both local backends to maintain per-visual GPU/Matplotlib state.
- The `clear()` method on `RendererBase` and the open TODO entry calling for `clear_viewport(viewport)` on the network renderer.

A diff-based render call (send only changed buffers, keyed by UUID) is implementable on top of the current Pydantic layer but is not yet implemented.

7. Security and Privacy

7.1 Threat model

The local backends (Matplotlib, Datoviz) execute entirely in-process and do not introduce a new threat surface beyond their underlying libraries.

The network backend introduces a server that:

1. Accepts arbitrary HTTP POST traffic on `/render`.
2. Deserializes a Pydantic-typed JSON payload (including base64-encoded buffers).
3. Constructs and renders the scene using either Matplotlib or an offscreen Datoviz instance.
4. Returns the resulting PNG.

Plausible adversaries:

- **Network attackers** with access to the server's port, attempting to crash, hang, or exfiltrate data through the rendering server.
- **Co-tenants** on a shared host who can reach the server's bind address.
- **Malicious clients** that intentionally send oversized or malformed payloads.

7.2 Concrete risks

- **No authentication.** The reference Flask server has no auth, no rate limiting, and no TLS termination. Anyone able to reach the port can render and consume server resources.
- **Resource exhaustion.** Buffers are sized by the client. Submitting a payload with a billion points will consume server memory, CPU, and (for Datoviz) GPU memory.
- **Deserialization surface.** Pydantic v2 mitigates classic type confusion, but the discriminated `PydanticVisual` union and base64 buffer handling are non-trivial parsers; their robustness depends on Pydantic and the GSP parser code.
- **Information disclosure in transit.** Scene data is sent unencrypted in JSON. For sensitive measurements, the body and the rendered PNG both carry information that should be protected in transit.
- **Server-side rendering side channels.** The server runs a real renderer; bugs in Matplotlib or Datoviz become server-side bugs.

7.3 Mitigations

The repository does not currently implement these; they are recommendations for any deployment beyond a single-user developer machine:

- Bind to `127.0.0.1` by default and require explicit configuration to expose externally.
- Terminate TLS at a reverse proxy.
- Add an authentication token (HMAC or bearer) on every request.
- Cap request size and total buffer count; reject scenes that exceed a configured budget.
- Run the renderer in a sandboxed worker process (separate user, resource limits) and enforce a per-render timeout.
- Log payload metadata (size, renderer choice, source IP) for auditing.

7.4 Privacy

Scene data — measured values, image data, model matrices — is conveyed verbatim to the rendering server. Operators handling sensitive data (medical imaging, proprietary measurements) must treat the server as a data-processing endpoint and apply the appropriate access controls.

8. Comparison with Existing Solutions

The defining axis is whether the API and the renderer are separable.

System	Scene API	Renderer(s)	Separable	Remote path
Matplotlib	Figure/Axes	Agg (CPU), TkAgg, etc.	No	None native
Datoviz	App/Figure/Panel	Vulkan (GPU)	No	None
VisPy	SceneGraph	OpenGL	No	Limited
Plotly / Bokeh	JSON figures	Browser (WebGL/2D)	No	Browser-coupled
Three.js + Py	Three.js scene	WebGL	No	Web-coupled
GSP_API	GSP scene model	Matplotlib, Datoviz, Network	**Yes**	Built-in (Flask)

The qualitative differences:

- Matplotlib has the largest installed base and the richest 2D feature set; it has no GPU path and no native remote path.
- Datoviz delivers the highest performance on a single workstation but is single-renderer and currently OS-restricted.
- VisPy offers a multi-canvas scene graph but the canvas implementation is OpenGL-bound; the scene graph is not a renderer-agnostic API in the GSP sense.
- Plotly and Bokeh excel at browser-delivered plots but their scene model and runtime are tied to the browser.
- GSP_API does not compete with any of these on rendering quality or on completeness of features. It competes on a different axis: whether the same scene description can target three distinct rendering regimes.

The uniqueness claim is structural, not aesthetic.

9. Use Cases

Backend portability for plotting libraries. A plotting library author writes against GSP_API once. End users select Matplotlib for compatibility and publication, Datoviz for interactive exploration of large datasets, or the network renderer for headless cluster jobs — without the author maintaining three implementations.

Headless figure generation. CI pipelines and HPC batch jobs render scenes through `gsp_network` or offscreen `gsp_datoviz` to PNGs for inclusion in reports, papers, or dashboards, without provisioning a display.

Remote GPU rendering. A researcher on a laptop without a GPU sends scene descriptions to a Datoviz server on a workstation or cluster node and receives rendered images locally, with no client-side GPU dependency.

Comparative renderer studies. Researchers studying rendering API design (an explicit project goal in TODO.md: "design principles for rendering library API with multiple backends") use GSP_API as a fixed scene specification against which CPU, GPU, and network rendering implementations can be measured side by side.

Teaching scenes once, demonstrating thrice. Educational material can demonstrate the same scientific scene under three rendering regimes from a single script, illustrating the practical effect of the renderer choice on speed, fidelity, and deployment.

Session capture for reproducibility. The planned session record/replay subsystem (Section 10) is intended to capture timestamped GSP commands and replay them on a remote machine for reproducible visualization workflows.

10. Limitations and Future Work

GSP_API is at version 0.1.0; the `description` field of `pyproject.toml` is empty; there is no published Git remote referenced in the working tree, and no public documentation URL is declared. The maturity level is research prototype, not production library.

10.1 Known issues from TODO.md

- **Datoviz text rendering.** Text anchor and rotation cannot currently be set in the Datoviz backend; the cause is not yet identified.
- **Network renderer clearing.** The network renderer erases the whole canvas on every call, which breaks multi-call rendering patterns (e.g., axes-managed examples). A `clear()` or `clear_viewport(viewport)` API is the proposed fix and is open.
- **Geometry sanity check.** The `sanity_check()` method on `Geometry` has an unresolved inheritance issue affecting `MeshGeometry`.

10.2 In-flight work

- Active feature branch `mesh_include_2` extends mesh handling.
- Collaborator sandboxes (`examples_nico/` , `src/gsp_nico/`) carry experimental code that has not been promoted into the public API.
- About 17 work-in-progress examples (underscore-prefixed) exercise unstable surfaces, including pyramid-loading examples, multi-renderer testers, and mesh variants.

10.3 Planned

- A transform DSL with unit-aware coordinates (NDC, px, cm, inch, pt) and viewport-relative coordinate transforms.
- A session record/replay subsystem capable of capturing timestamped GSP commands and replaying them, intended for reproducibility and remote workflows.
- A multi-viewport, multi-renderer test harness ("big tester") covering matrix combinations of renderers, viewports, and serialization variants.
- Resolution-adaptive image-pyramid loading for large image data.
- Transparent-background support in the network renderer (currently only the Matplotlib local backend supports transparent backgrounds end-to-end).
- Datoviz feature-parity work for text rendering.
- An incremental rendering protocol on top of the network renderer (no design exists yet; current architecture admits one).

10.4 Adoption constraints

The Python version constraint (`>=3.13,<4.0`) is aggressive and currently limits installation in most working environments. Until Python 3.13 is more widely available, the practical user base is small.

11. Conclusion

The strategic significance of GSP_API is structural rather than competitive. It does not aim to be faster than Datoviz, broader than Matplotlib, or prettier than Plotly. It aims to be the first piece of API code that does not need to be rewritten when the renderer changes. The cost of that abstraction is an honest one — feature ceilings are set by the lowest common denominator of the three backends, performance is bounded by the chosen renderer, and the network path lacks an incremental protocol — but the cost is paid once, in the API design, instead of repeatedly, in every program that wants to retarget.

For engineers building plotting tools on top of the scientific Python stack, GSP_API offers a portability layer that converts a backend choice from a code rewrite into a string. For researchers studying rendering-library API design, it offers a substrate against which CPU, GPU, and network implementations can be compared inside a fixed specification. The current 0.1.0 prototype is incomplete in both feature parity and operational maturity, but the separation it draws between scene description and renderer is the contribution worth evaluating.

Appendix A: Tech Stack

- **Language and runtime.** Python `>=3.13,<4.0` .
- **Core dependencies.** numpy `>=2.3.4` , matplotlib `>=3.10.7` , pydantic `>=2.12.4` , datoviz `>=0.3.2,<0.4` , flask, imageio, loguru, colorama, requests-file, http-constants, mpl3d (`git+https://github.com/rougier/matplotlib-3d`).
- **Build and tooling.** Poetry (`poetry-core>=2.0`), pytest `>=8.4` , mypy `>=1.18` , ruff `>=0.14.9` , black (line length 160), pydoclint, mkdocs-material with mkdocstrings, pre-commit.

Appendix B: Stated Assumptions

- This document is grounded in the state of the repository as observed in the working tree. Where the project's own marketing summary disagreed with the source — most notably the claim that backend selection is performed via a `GSP_RENDERER` environment variable, which is not the case — this paper follows the source.
- No performance benchmarks are published in the repository, so the performance discussion is qualitative.

- The repository has no `remote.origin.url` configured locally and no public documentation URL declared; references to those URLs are deliberately omitted rather than guessed.